Dockerfile. Создание собственных образов

От исходного кода Express.js до Docker-образа

Цель и результаты лекции

- Понимать назначение Dockerfile и контекста сборки.
- Различать инструкции сборки и запуска контейнера.
- Создавать образ минимального Express API.
- Использовать .dockerignore и кэш зависимостей.
- Проверять работу и устранять ошибки сборки.

Что мы уже умеем

- Из лекции 1: образ — шаблон, контейнер — экземпляр.
- Из лекции 2: docker run, ps, logs, exec, stop и rm.
- Из лекции 3: pull, image ls, history, inspect, tag и реестры.
- Теперь создадим собственный образ из приложения.

Учебные вопросы

№	Учебный вопрос
1	Dockerfile, контекст сборки и образ
2	Инструкции Dockerfile
3	Express API и первый Dockerfile
4	Сборка, запуск и диагностика
5	Кэш, .dockerignore и качество образа

Dockerfile и контекст сборки

Как из исходного кода получается образ

Почему одного исходного кода недостаточно

- Приложению нужна подходящая версия Node.js.
- Нужны внешние зависимости из package.json.
- Требуется выбрать рабочий каталог и команду запуска.
- Одинаковое описание сборки помогает воспроизводить среду.

Определение Dockerfile

- Dockerfile — текстовый файл с инструкциями построения образа.
- Инструкции выполняются сборщиком последовательно.
- Образ создаётся командой docker build.
- Стандартное имя файла — Dockerfile без расширения.

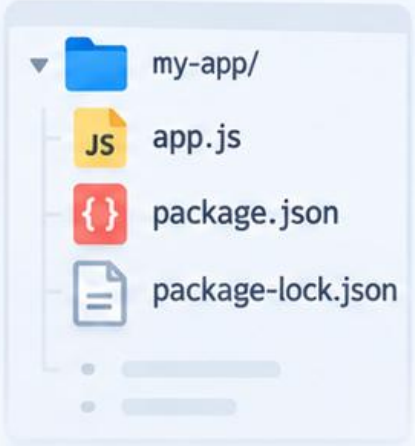
Главная идея

Dockerfile описывает сборку образа, а docker run создаёт контейнер.

От исходного кода к Docker-образу

Как файлы приложения превращаются в готовый Docker-образ

Исходный код приложения



Файлы исходного кода и конфигурации проекта.

Dockerfile

```
FROM node:22
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

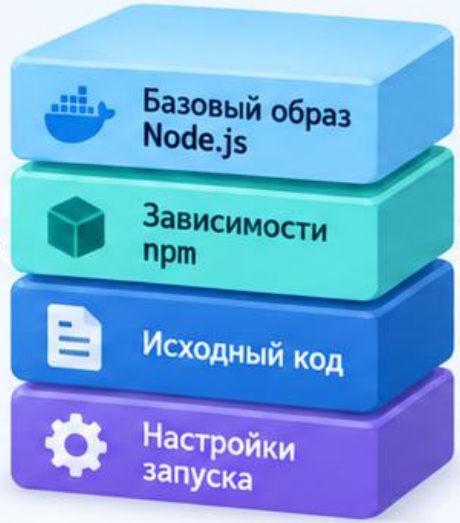
Файл Dockerfile описывает, как собрать Docker-образ из файлов приложения.

Сборка образа

```
$ docker build -t
  express-api:v1 .
```

Docker читает Dockerfile и выполняет инструкции по порядку.

Слои образа



Каждая инструкция Dockerfile обычно создаёт новый слой в образе.

Готовый Docker-образ



Образ можно запускать много раз.

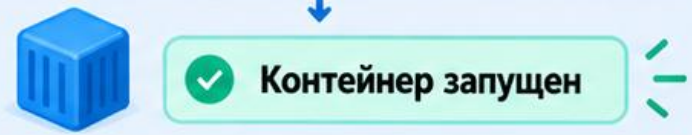
Что важно понять

- Dockerfile описывает процесс сборки.
- **docker build** создаёт образ из контекста проекта.
- Каждый шаг Dockerfile обычно создаёт новый слой.
- Образ — это шаблон для запуска контейнеров.

Следующий шаг

Запустим контейнер из готового образа:

```
$ docker run -p 3000:3000 express-api:v1
```



Этапы превращения проекта в контейнер

1	**Шаг 1**	Подготовить исходный код и package.json.
2	**Шаг 2**	Создать Dockerfile и .dockerignore.
3	**Шаг 3**	Выполнить docker build с контекстом сборки.
4	**Шаг 4**	Проверить образ через docker image ls.
5	**Шаг 5**	Создать контейнер с docker run и проверить HTTP-ответ.

Контекст сборки

- Контекст — набор файлов, доступных сборщику.
- В команде **docker build -t api:v1 .** точка задаёт текущий каталог как контекст.
- COPY берёт исходные файлы из контекста.
- Файлы вне контекста нельзя произвольно копировать через COPY.

Команда сборки образа

```bash
docker build -t express-api:v1 .
```

Что важно

- build — сборка образа по Dockerfile.
- -t задаёт имя и тег образа.
- Точка в конце — текущий каталог как контекст.
- Ожидаемый результат: успешная сборка и образ в docker image ls.

Основные инструкции Dockerfile

Что выполняется при сборке и при запуске

Базовые инструкции Dockerfile

Инструкция	Назначение	Пример
FROM	Базовый образ	FROM node:22-alpine
WORKDIR	Рабочий каталог	WORKDIR /app
COPY	Копирование файлов	COPY . .
RUN	Команда во время сборки	RUN npm ci
EXPOSE	Документирование порта	EXPOSE 3000
CMD	Команда по умолчанию при запуске	CMD ["node","app.js"]

FROM: выбираем базовый образ

DOCKERFILE

```
FROM node:22-alpine
```

Что важно

- Используем образ с Node.js 22 на Alpine Linux.
- Базовый образ содержит среду выполнения.
- Тег может обновляться: для воспроизводимости в production закрепляют digest.

WORKDIR: задаём рабочий каталог

```
DOCKERFILE
WORKDIR /app
```

Что важно

- Устанавливает рабочий каталог последующих инструкций.
- Каталог создаётся, если его нет.
- Предпочтительнее не полагаться на каталог по умолчанию.

COPY: переносим файлы

```
DOCKERFILE
COPY package*.json ./
COPY . .
```

Что важно

- Первая инструкция копирует манифест и lock-файл.
- Вторая переносит остальные файлы проекта из контекста.
- При наличии .dockerignore ненужные файлы исключаются.

RUN: выполняем команду при сборке

DOCKERFILE

```
RUN npm ci --omit=dev
```

Что важно

- Устанавливает зависимости в образ во время сборки.
- npm ci требует существующего согласованного package-lock.json.
- --omit=dev не устанавливает devDependencies на диск.

EXPOSE: документируем порт

```
DOCKERFILE
EXPOSE 3000
```

Что важно

- Указывает порт, на котором предполагается работа приложения.
- Не делает порт доступным на хосте автоматически.
- Для публикации при запуске используйте -p 3000:3000.

CMD: задаём основной процесс

```dockerfile
CMD ["node", "app.js"]
```

Что важно

- JSON/exec-форма запускает node без промежуточной оболочки.
- Команда по умолчанию может быть переопределена в docker run.
- После завершения основного процесса контейнер останавливается.

RUN и CMD — принципиальное различие

RUN

- Исполняется во время docker build.
- Выполняет операции подготовки образа.
- Пример: RUN npm ci.

CMD

- Определяет команду запуска контейнера.
- Выполняется при docker run, а не при docker build.
- Пример: CMD ["node", "app.js"].

CMD и ENTRYPOINT

- CMD задаёт аргументы или команду по умолчанию.
- ENTRYPOINT определяет основной исполняемый файл контейнера.
- В этой лекции применяем только CMD; ENTRYPOINT рассмотрен обзорно.
- Оба поддерживают exec-форму с массивом строк.

Инструкции Dockerfile — сборка и запуск

Основные инструкции Dockerfile и как они превращают исходный код в готовый контейнер

1 Пример Dockerfile (Node.js / Express)

```
1  FROM node:20-alpine
2  WORKDIR /app
3  COPY package*.json ./
4  RUN npm install
5  COPY . .
6  EXPOSE 3000
7  CMD ["node", "app.js"]
```

FROM
Базовый образ, с которого начинается сборка (например, `node:20-alpine`).

WORKDIR
Устанавливает рабочую директорию внутри образа.

COPY package*.json ./
Копирует файлы зависимостей.
Делается отдельно для использования кэша.

RUN
Выполняет команду во время сборки.
Здесь устанавливаются зависимости (`npm install`).

COPY . .
Копирует остальные файлы приложения в образ.

EXPOSE
Документирует порт, который будет использовать приложение (не публикует его).

CMD
Команда, которая запускается при старте контейнера.

2 Сборка образа

docker build -t myapp:1.0 .



Контекст сборки
(текущая папка)

- app.js
- package.json
- Dockerfile
- .dockerignore
- ...

```
docker build -t myapp:1.0 .
```

Сборка образа (создание слоёв)



Слой 7: CMD
Слой 6: EXPOSE
Слой 5: COPY . .
Слой 4: RUN npm install
Слой 3: COPY package*.json
Слой 2: WORKDIR
Слой 1: FROM node:20-alpine



Готовый образ
myapp:1.0

3 Запуск контейнера

docker run -d -p 3000:3000 myapp:1.0

```
docker run -d -p 3000:3000 myapp:1.0
```



Запущенный контейнер
myapp:1.0

Порт контейнера:
3000 (EXPOSE)

Порт хоста:
3000 (-p 3000:3000)

http://localhost:3000

```
{
  "message": "Hello from Docker!",
  "version": "1.0"
}
```

Приложение доступно на хосте
http://localhost:3000

Результат

Исходный код упакован в Docker-образ, из которого запускается готовое приложение в изолированном контейнере.

Создаём минимальный Express API

Практический пример для последующей контейнеризации

Структура проекта

- Каталог express-api/.
- app.js — HTTP-сервер.
- package.json и package-lock.json — зависимости.
- Dockerfile — инструкция построения образа.
- .dockerignore — исключения из контекста.

Инициализация приложения

```javascript
mkdir express-api
cd express-api
npm init -y
npm install express
```

Что важно

- Эти команды выполняются на компьютере разработки с Node.js/npm.
- После npm install возникает package-lock.json.
- Для Docker в Debian можно подготовить файлы заранее, без локальной установки Node.js.

Минимальный app.js

```javascript
import express from "express";

const app = express();
app.get("/api/health", (req, res) => {
  res.json({ status: "ok" });
});
app.listen(3000, "0.0.0.0", () => {
  console.log("API listening on port 3000");
});
```

Что важно

- GET /api/health возвращает JSON для проверки.
- 0.0.0.0 позволяет принимать подключения на интерфейсах контейнера.
- В package.json нужно указать "type": "module" для import.

Что необходимо в package.json

- Установленная зависимость express в dependencies.
- Поле "type": "module" для ES Modules.
- Скрипт "start": "node app.js" удобен, но не обязателен для CMD.
- Файл package-lock.json необходим для npm ci.

Пример Dockerfile целиком

DOCKERFILE

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Что важно

- Базовый образ с Node.js.
- Зависимости устанавливаются до копирования кода ради кэша.
- EXPOSE лишь документирует порт.
- Контейнер запускает node app.js.

Как читать Dockerfile построчно

1	**Шаг 1**	FROM: взять базовый образ.
2	**Шаг 2**	WORKDIR: установить /app.
3	**Шаг 3**	COPY package*.json: перенести манифест и lock-файл.
4	**Шаг 4**	RUN npm ci: установить зависимости.
5	**Шаг 5**	COPY . .: скопировать код приложения.
6	**Шаг 6**	EXPOSE: указать используемый порт.
7	**Шаг 7**	CMD: задать команду запуска.

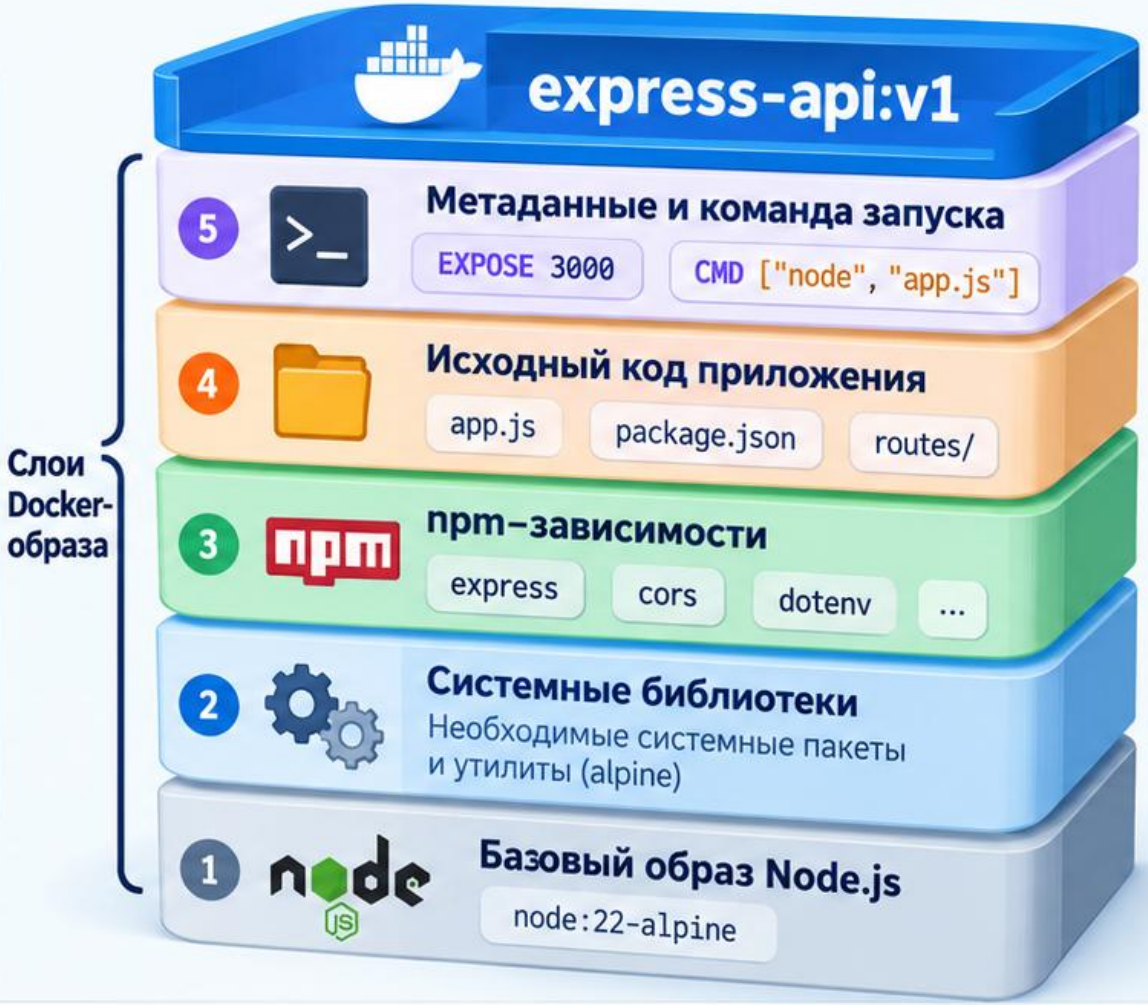
Структура образа Express API

Что входит в Docker-образ Node.js / Express-приложения

Dockerfile формирует образ

Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```



Что получает контейнер при запуске

Среду Node.js
Node.js версии 22 (alpine)

Установленные зависимости
Например: express, cors, dotenv

Код Express API
Файлы приложения: app.js, package.json, routes/

Команду запуска приложения
Запускается: node app.js



Важно понимать

1. Образ — шаблон для контейнера.
2. Слои образа обычно неизменяемы.
3. Один образ можно запускать много раз.
4. Порт публикуется отдельно через -p.

Сборка, запуск и диагностика

От docker build до ответа HTTP

Собираем свой образ

```bash
docker build -t express-api:v1 .
```

Что важно

- Запускать из каталога проекта с Dockerfile.
- Проверьте завершение сборки без ошибок.
- Проверьте новый образ командой docker image ls.

Проверяем наличие образа

BASH

```
docker image ls
docker image inspect express-api:v1
```

Что важно

- В списке появится express-api с тегом v1.
- inspect покажет метаданные образа.
- Команды не запускают контейнер.

Запуск и публикация порта

```bash
docker run -d --name api1 -p 3000:3000 express-api:v1
```

Что важно

- -d запускает в фоновом режиме.
- --name присваивает имя контейнеру.
- -p 3000:3000 связывает порт хоста с портом контейнера.

Проверка HTTP API

```bash
curl http://localhost:3000/api/health
```

Что важно

- Ожидаемый ответ: {"status":"ok"}.
- Команда выполняется на хосте с Docker.
- Если браузер на Windows, а Docker внутри Debian VM, используйте доступный IP VM.

Проверка состояния и логов

BASH

```
docker ps
docker logs api1
docker inspect api1
```

Что важно

- ps показывает запущенные контейнеры.
- logs покажет вывод Node.js.
- inspect помогает проверить состояние и публикацию портов.

Остановка и удаление

BASH

```
docker stop api1
docker rm api1
```

Что важно

- stop отправляет сигнал завершения основному процессу.
- rm удаляет остановленный контейнер.
- Образ express-api:v1 при этом сохраняется.

Сборка и запуск собственного образа

Как из Dockerfile получить свой образ и запустить контейнер с приложением

1. Иссодный проект

Подготавливаем файлы приложения.

express-api/

- app.js
- package.json
- package-lock.json
- Dockerfile
- .dockerignore

2. Dockerfile

Описывает, как собрать образ.

Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

3. Сборка образа

Docker читает Dockerfile и создаёт слои образа.

```
$ docker build -t
  express-api:v1 .
```

- Чтение Dockerfile
- Создание слоёв образа
- Готовый образ с тегом express-api:v1

4. Готовый образа

Результат сборки — Docker-образ.



- Можно запускать много раз.
- Содержит Node.js, зависимости и код приложения.

5. Запуск контейнера

Создаём и запускаем контейнер из образа.

```
$ docker run -d \
  --name api1 \
  -p 3000:3000 \
  express-api:v1
```

`-d`	Запуск в фоновом режиме (detached mode).
`--name`	Задаёт имя контейнера (в примере: api1).
`-p 3000:3000`	Публикация порта: порт 3000 контейнера доступен на порту 3000 вашей машины.

6. Контейнер работает

Приложение запущено и доступно по порту 3000.



▶ Контейнер запущен

http://localhost:3000

```
{
  "message": "Hello from
  Docker!"
}
```

Что важно понять

docker build создаёт образ.
Читает Dockerfile и формирует многослойный образ.

docker run создаёт и запускает контейнер.
Из указанного образа Docker создаёт контейнер и запускает его.

Образ — шаблон, контейнер — его экземпляр.
Из одного образа можно запустить много контейнеров.

Порт EXPOSE не публикует доступ сам по себе — нужен -p.
EXPOSE только указывает порт внутри контейнера. Чтобы сделать его доступным снаружи, нужно использовать -p.

Что делать при ошибке подключения

- Проверить docker ps: контейнер действительно работает?
- Посмотреть docker logs api1: нет ли ошибки Node.js?
- Проверить сопоставление 3000:3000.
- Проверить, что Node.js слушает 0.0.0.0.
- Проверить правильность URL, IP машины и сетевых настроек VM.

Кэш и качество сборки

Уменьшаем число лишних операций

Как работает кэш сборки

- Сборщик может повторно использовать результаты неизменившихся шагов.
- Изменение исходного кода не должно каждый раз переустанавливать npm-пакеты.
- Для этого сначала копируем package*.json и выполняем npm ci.
- Потом копируем остальные файлы проекта.

Порядок COPY и использование кэша

Неудачный порядок

- COPY . .
- RUN npm ci
- Любое изменение кода может сделать кэш установки зависимостей непригодным.

Предпочтительный порядок

- COPY package*.json ./
- RUN npm ci --omit=dev
- COPY . .
- Изменение только app.js обычно сохраняет кэш установки зависимостей.

Назначение .dockerignore

- Исключает ненужные файлы из контекста сборки.
- Убирает локальные node_modules и служебные каталоги.
- Помогает не переносить случайно секреты в контекст.
- Не заменяет полноценную защиту секретов.

Файл .dockerignore

```text
node_modules
npm-debug.log
.git
.env
coverage
```

Что важно

- Каждая строка задаёт шаблон исключения.
- node_modules не переносим с хоста: зависимости устанавливаем в образе.
- .env не копируем в образ; секреты не храним в исходном коде.

Кэширование слоёв и .dockerignore

Как Docker ускоряет повторную сборку и зачем исключать лишние файлы из контекста

Кэширование слоёв Docker

Если инструкции и её входные данные не изменились, Docker использует кэш слоя.

Dockerfile (Express / Node.js)

1. **FROM node:22-alpine** — Базовый образ Node.js
2. **WORKDIR /app** — Рабочая директория
3. **COPY package*.json ./** — Копируем файлы зависимостей
4. **RUN npm install** — Устанавливаем зависимости
5. **COPY . .** — Копируем остальной код
6. **CMD ["node", "app.js"]** — Команда по умолчанию

Первая сборка

Все шаги выполняются полностью

FROM node:22-alpine	Скачивается образ — rebuild
WORKDIR /app	Создаётся слой — rebuild
COPY package*.json ./	Копируются файлы — rebuild
RUN npm install	Устанавливаются зависимости — rebuild
COPY . .	Копируется код приложения — rebuild
CMD ["node", "app.js"]	Создаётся финальный слой — rebuild

Повторная сборка

Изменён только app.js

FROM node:22-alpine	Используется кэш — cache
WORKDIR /app	Используется кэш — cache
COPY package*.json ./	Используется кэш — cache
RUN npm install	Устанвзуется кэш — cache
COPY . .	Копируется обновлённый код — rebuild
CMD ["node", "app.js"]	Обновляется финальный слой — rebuild

Если инструкция и её входные данные не изменились, Docker **использует кэш слоя.**

Изменение **package.json** обычно **ломает кэш для npm install.**

Изменение **app.js** не требует повторной установки зависимостей, если зависимости не менялись.

```
$ docker build -t express-api:v1 .        # Команда сборки образа
```

.dockerignore

Исключает лишние файлы из контекста сборки.

Структура проекта

```
express-api/
  app.js
  package.json
  package-lock.json
  routes/
  Dockerfile
  node_modules/
  .git/
  logs/
  .env
  dist/
  coverage/
```

Попадает в build context

- app.js
- package.json
- package-lock.json
- routes/
- Dockerfile

Исключается через .dockerignore

- node_modules/
- .git/
- logs/
- .env
- dist/
- coverage/

Содержимое файла .dockerignore

```
node_modules
.git
.env
logs
coverage
dist
```

.dockerignore **уменьшает build context.**

Лишние файлы не отправляются Docker при сборке.

Это **ускоряет сборку** и уменьшает вероятность случайно включить секреты и мусорные файлы.

Что важно понять

1. Порядок инструкций Dockerfile влияет на кэширование.
2. Сначала копируют package*.json, потом устанавливают зависимости, потом копируют остальной код.
3. **.dockerignore** сокращает контекст сборки.
4. Грамотный Dockerfile делает сборку быстрее и чище.

Типичные ошибки начинающих

- Забыли package-lock.json — npm ci выдаст ошибку.
- Забыли "type": "module" — import не обработается как ES Module.
- EXPOSE 3000 без -p — с хоста сервис не опубликован.
- Приложение слушает только 127.0.0.1 внутри контейнера.
- Неверно выбран контекст сборки или имя Dockerfile.

Подготовка к сквозному проекту

- Сегодня создан образ Express API без базы данных.
- В следующей теме научимся сохранять данные в volumes.
- Позже подключим PostgreSQL и Nginx через Docker Compose.
- Тот же образ API станет частью итогового проекта.

Итоги лекции

1. Dockerfile описывает шаги построения собственного образа.
2. RUN выполняется при сборке, CMD задаёт запуск контейнера.
3. Сборка: docker build -t имя:тег .; запуск: docker run.
4. EXPOSE не публикует порт: используйте -p.
5. .dockerignore и порядок COPY помогают управлять контекстом и кэшем.

Контрольные вопросы

1. 1. Что такое контекст сборки и зачем в docker build ставят точку?
2. 2. Чем RUN отличается от CMD?
3. 3. Почему EXPOSE 3000 не делает API доступным через localhost?
4. 4. Зачем копировать package*.json отдельно от app.js?
5. 5. Почему npm ci требует package-lock.json?
6. 6. Для чего нужен .dockerignore?
7. 7. Какие команды проверят образ, контейнер и HTTP-ответ?